**SIMATS ENGINEERING**

**ASSESSMENT TOOL 2**

**COURSE NAME:** Artificial Intelligence **COURSE CODE:** CSA17

COURSE OUTCOME COVERED

**CO1:** Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.( BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

**Annexure A**

**Constraint-Based Problem Solving**

Code of Conduct:

I __M.Greeshma/192524195__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

**Signature of the student:** M.Greeshma

**1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night)**

**such that:**

**Constraints:**

**i. D1 cannot work Night shift**

**ii. D2 must work before D3 (Morning < Afternoon < Night)**

**iii. Only one doctor per shift**

**iv. D3 cannot work Morning**

**v. All doctors must be assigned exactly one shift**

**Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule**

**Hospital Doctor Shift Assignment Using Backtracking Search**

**Problem:** Assign three doctors (D1, D2, D3) to Morning, Afternoon, and Night shifts while satisfying all the given constraints.

**A. Variables and domains**

- Variables: D1, D2, D3
- Domain:

**B. Constraints**

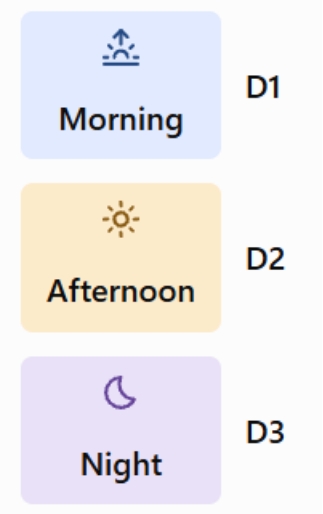
1. D1 cannot work Night: $D1 \ne N$
2. D2 must work before D3: $D2 < D3$, where $M < A < N$.
3. Only one doctor per shift.
4. D3 cannot work Morning: $D3 \ne M$.
5. Every doctor must be assigned exactly one shift.

**C. Backtracking search with intermediate steps**

We assign shifts in the order D1 → D2 → D3 and check the constraints after each assignment.

| Step | Assignment | Constraint check | Result |
|---|---|---|---|
| 1 | D1 = Morning | D1 is not assigned Night | Valid |
| 2 | D2 = Morning | Morning is already assigned to D1 | Reject |
| 3 | D2 = Afternoon | D2 is later than D1, but D2 must be before D3 | Continue |
| 4 | D3 = Morning | D3 cannot work Morning; Morning is occupied | Reject |
| 5 | D3 = Afternoon | Afternoon is occupied by D2 | Reject |
| 6 | D3 = Night | D2 = Afternoon is before D3 = Night; shifts are unique | Valid |

**D. Final valid schedule**




Result: D1 = Morning, D2 = Afternoon, D3 = Night. All five constraints are satisfied.

Backtracking explores possible assignments and rejects any partial assignment that violates a constraint. If a conflict occurs, it returns to the previous variable and tries another value.

**2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G).**

**Obstacles block certain paths.**

**Constraints:**

**i. Movement allowed: Up, Down, Left, Right**

**ii. Cost of each move = 1**

**iii. Cannot pass through obstacles**

**iv. Use Manhattan Distance heuristic**

**Using above constraints and BFS Search Algorithm Show step-by-step node expansion**

**and priority queue updates and determine the optimal path and cost.**

**Robot Navigation in a 5×5 Grid**

**Problem:** A robot must move from Start (S) to Goal (G), avoiding obstacles and finding the shortest path.

Since the question does not specify the locations of S, G, or the obstacles, the following grid is used as an example.

**A. Grid representation**

- Start: (1,1)
- Goal: (5,5)
- Obstacles: (2,2), (3,2), and (4,4)
- Allowed movements: Up, Down, Left, Right
- Cost of each movement: 1

**B. Manhattan distance heuristic**

Manhattan distance estimates the distance from a cell to the goal:

$$h(n)=|x_n-x_G|+|y_n-y_G|$$

For example, at the start position (1,1):

$$h(S)=|1-5|+|1-5|=8$$

This heuristic estimates the minimum number of moves when movement is restricted to four directions and ignores obstacles.

**C. Step-by-step BFS node expansion**

Breadth-First Search (BFS) uses a FIFO queue and explores nodes level by level. The following trace uses Right, Down, Left, Up as the neighbor order.

| Step | Expanded node | New valid nodes added to queue |
|---|---|---|
| 1 | (1,1) | (1,2), (2,1) |
| 2 | (1,2) | (1,3) |
| 3 | (2,1) | (3,1) |
| 4 | (1,3) | (1,4), (2,3) |
| 5 | (3,1) | (4,1) |
| 6 | (1,4) | (1,5), (2,4) |
| 7 | (2,3) | (3,3) |
| 8 | (4,1) | (5,1), (4,2) |
| 9 | (1,5) | (2,5) |
| 10 | (2,4) | (3,4) |
| 11 | (3,3) | — |
| 12 | (5,1) | (5,2) |
| 13 | (4,2) | (4,3) |

| | | |
|---|---|---|
| 14 | (2,5) | (3,5) |
| 15 | (3,4) | (4,4) is blocked |
| 16 | (5,2) | (5,3) |
| 17 | (4,3) | — |
| 18 | (3,5) | (4,5) |
| 19 | (5,3) | (5,4) |
| 20 | (4,5) | (5,5) |
| 21 | (5,4) | — |
| 22 | (5,5) | Goal reached |

Previously visited cells are not added again. The queue holds all discovered but not yet expanded nodes, and nodes at smaller depths are expanded first.

**D. Optimal path and cost**

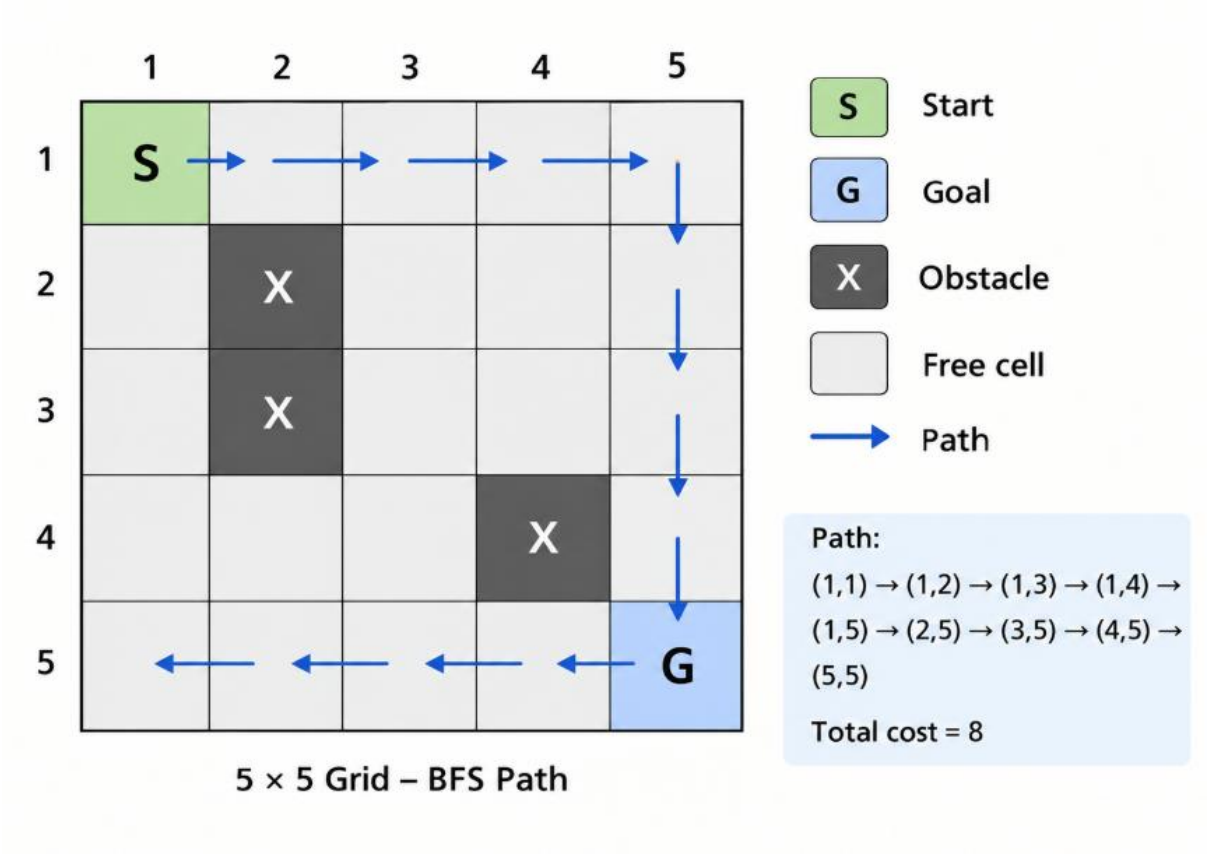
One shortest path found by BFS is:

$$\begin{aligned} S &\rightarrow (1,2)\rightarrow(1,3)\\ &\rightarrow(1,4)\rightarrow(1,5)\\ &\rightarrow(2,5)\rightarrow(3,5)\\ &\rightarrow(4,5)\rightarrow G \end{aligned}$$

There are eight movements, each costing 1.

$$\boxed{\text{Total path cost}=8}$$

Important distinction: BFS uses a FIFO queue and does not use Manhattan distance to decide which node to expand. A* Search uses a priority queue ordered by $f(n)=g(n)+h(n)$, where $g(n)$ is the cost so far and $h(n)$ is the Manhattan heuristic. Both BFS and A* find an optimal path here because all movement costs are equal and the heuristic is admissible.




**3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information. The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.**

**Constraints:**

**□ The robot can move up, down, left, right**

**□ Some cells are blocked (obstacles) and cannot be traversed**

**□ The robot has limited sensing capability (can only observe adjacent cells)**

**□ Each movement has a cost of 1, but entering risky zones has an additional cost of 2**

**□ The robot must minimize total cost while ensuring safe navigation**

**□ The robot must update its knowledge dynamically (online search scenario)**

**Tasks:**

**a) Analyze all the given constraints and explain how they affect the robot's decision-making process.**

**b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved.**

**c) Demonstrate how the robot applies AI concepts such as:**

**□ Intelligent agents**

**□ Rationality**

**□ Environment type**

**d) Justify why your chosen approach is the most suitable for solving this problem under the**

**given constraints.**

**Autonomous Rescue Robot in a Disaster-Affected Building**

**a) Analysis of constraints and their effects**

The robot must locate a survivor in an environment where some routes are blocked and information is incomplete.

| **Constraint** | **Effect on decision-making** |
|---|---|
| Movement in four directions | The robot can move only up, down, left, or right. |
| Obstacles | Blocked cells must be avoided. |
| Limited sensing | Only adjacent cells can be observed, so the robot cannot know the entire map initially. |

| | |
|---|---|
| Normal movement cost = 1 | Every ordinary move adds one unit to the path cost. |
| Risky-zone penalty = 2 | Entering a risky cell costs \(1+2=3\) units. |
| Minimum total cost | The robot should compare alternative routes and choose the lowest-cost safe route. |
| Dynamic knowledge updates | The robot must update its map when it discovers new obstacles, risks, or traversable cells. |

The robot must balance three objectives: reaching the survivor, minimizing movement and risk costs, and maintaining safe navigation.

**b) Solution approach using Uniform Cost Search and an online search agent**

**Chosen approach:** Combine an online search agent for exploration with Uniform Cost Search (UCS) for cost-aware route planning.

UCS is suitable because movements have non-negative costs, and risky cells have higher costs than normal cells.

Step 1: Initialize

- Record the starting position S.
- Set the initial path cost to zero.
- Add S to a priority queue.
- Mark unknown cells as unexplored.

Step 2: Sense the surroundings

Observe the adjacent cells and identify whether they are free, blocked, or risky.

Step 3: Generate possible actions

Consider moving up, down, left, or right. Reject any action that enters a known obstacle.

Step 4: Calculate movement costs

$$ c(n,n')= \begin{cases} 1, & \text{normal cell}\\ 3, & \text{risky cell} \end{cases} $$

Calculate the cumulative path cost:

$$ g(n')=g(n)+c(n,n') $$

Step 5: Update the priority queue

Insert or update reachable cells using their cumulative costs. Expand the cell with the lowest cost first.

Step 6: Update knowledge

After moving, observe adjacent cells again and update the internal map. If a newly discovered obstacle blocks the planned route, replan using the updated information.

Step 7: Reach the survivor

Stop when the survivor's location G is reached. The final route should have the lowest achievable total cost based on the available map and risk information.

**Pseudocode**

```
RESCUE_ROBOT(S, G):

    known_map = initialize_unknown_map()
    priority_queue = [(0, S, [S])]
    best_cost[S] = 0

    WHILE priority_queue is not empty:

        (cost, current, path) = POP_LOWEST_COST()

        IF cost > best_cost[current]:
```

```
    CONTINUE

IF current == G:
    RETURN path, cost

OBSERVE_ADJACENT_CELLS(current)
UPDATE_KNOWN_MAP()

FOR each neighbor of current:

    IF neighbor is blocked:
        CONTINUE

    IF neighbor is risky:
        move_cost = 3
    ELSE:
        move_cost = 1

    new_cost = cost + move_cost

    IF neighbor is unvisited
       OR new_cost < best_cost[neighbor]:

        best_cost[neighbor] = new_cost
        PUSH(new_cost, neighbor, path + [neighbor])
```

```
RETURN "No known path to survivor"
```

**Note:** This is the cost-aware planning component. A real online implementation must interleave sensing and movement and replan as new information becomes available, rather than assuming the complete map is known in advance.

**c) Application of AI concepts**

**1. Intelligent agents**

The rescue robot is an intelligent agent because it perceives its surroundings through sensors, processes information, and performs actions to achieve the goal of rescuing the survivor.

**2. Rationality**

A rational agent chooses the action expected to maximize its performance according to the information available. The robot should avoid known obstacles, consider risk penalties, and choose the lowest-cost safe route. Because the environment is partially observable, it cannot guarantee that every decision is optimal with respect to information it has not yet discovered.

**3. Environment type**

- Partially observable: Only adjacent cells are visible.
- Stochastic or uncertain: The outcome of navigation may be uncertain in a damaged building.
- Sequential: Each action affects later decisions.
- Dynamic: New obstacles or hazards may be discovered during exploration.
- Discrete: The building is represented by individual grid cells and discrete movements.
- Single-agent: The robot is the primary decision-making agent**.**

**d) Justification for the chosen approach**

The combination of online exploration and UCS is appropriate for the following reasons:

1. Cost optimization: UCS considers cumulative movement and risk costs, unlike ordinary BFS, which assumes equal edge costs.
2. Safety: Known obstacles are excluded, and risky zones receive additional cost.
3. Adaptability: Online exploration lets the robot update its map when it discovers new information.
4. Rational decision-making: The robot can compare alternative routes instead of choosing the first available path.
5. Goal achievement: UCS can find a least-cost path to the survivor when the relevant traversable paths and their costs are known.

**Conclusion:** An online rescue agent using UCS for cost-aware planning can navigate an incompletely known environment, update its knowledge dynamically, avoid obstacles, and minimize total travel and risk costs. However, a real rescue system should also enforce hard safety limits; the cheapest route is not necessarily acceptable if it exposes the robot to unacceptable danger.